

ECE0202: Embedded Processors and Interfacing

Lab 2: Interfacing Push Button and LED (in C)

Due: TBD

Objectives

- Get familiar with KEIL uvision software development environment
- Create a C project for the Nucleo kit and program the kit
- Learn the basics of GPIO input and output configuration
- Perform simple digital I/O input (interfacing push button) and output (interfacing LED)
- Understand polling I/O (busy waiting) and its inefficiency

Pre-Lab Reading

- Read textbook chapter 4.6 to review bitwise operations.
- Read textbook chapter 14 to learn about GPIO interfacing.

Deliverables – total 100 points

- (50 points) A demonstration of the working pushbutton and LED combination to the TA.
- (30 points) A submission of your code with comments.
- (20 points) A submission of answers to the pre-lab questions.

Submit combine pre-lab answers and your code as a pdf file and submit it to Black board.

Getting Started

This project is written in the C language. Download the “C Project Template” from BB. If you load the project in KEIL, you will see a “main.c” file as part of the project. Open this file and write your project code in the the “USER CODE GOES HERE” area.

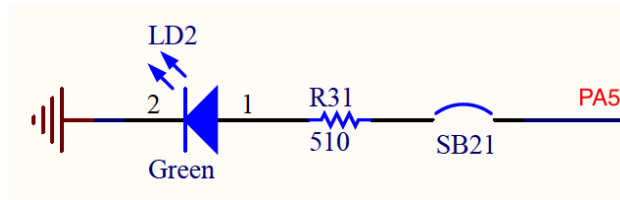
Lab Assignment

- Following Chapter 14, implement a C program that toggles the green LED when the user button is pressed. (It can be understood that long-pressing the push button will make the LED flash)

LEDs on the Board

Part A: Green LED

There are one user-controllable green LED (light-emitting diodes) on the STM32 NUCLEO-L476RG board. The LED one is connected to the pin GPIO **PA5** (Port A Pin 5), as shown in the figure below.



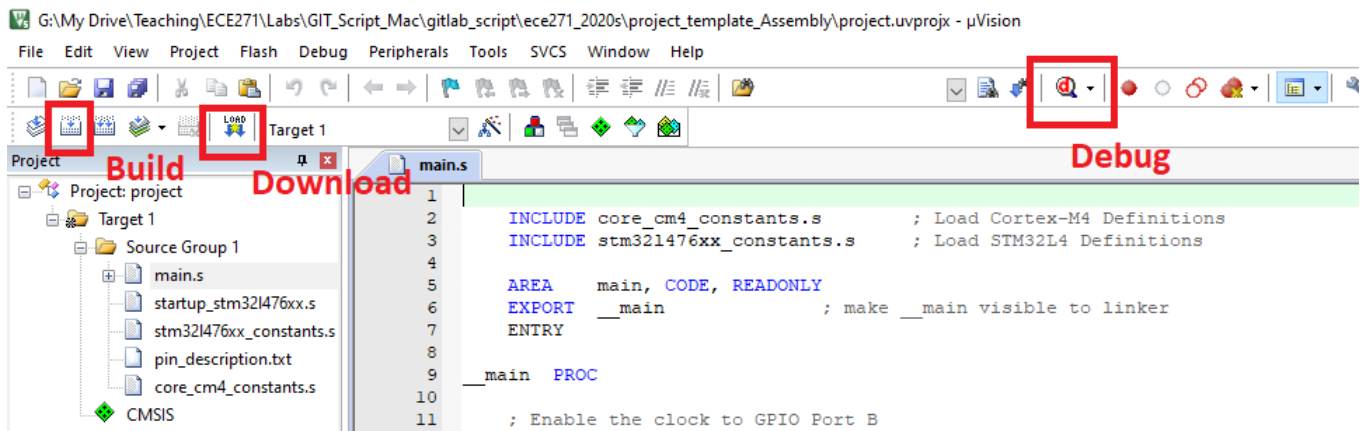
To turn on a LED, software must at least perform the following five operations:

1. **Enable the clock** of the corresponding GPIO port. By default, the clock to all peripherals, including GPIO ports, are turned off to save the energy.
2. Set the **mode** of the corresponding GPIO pin must be set as **output**. By default, the mode of all GPIO pin is analog.
3. Set the push-pull/open-drain setting for the GPIO pins to **push-pull**
4. Set the pull-up/pull-down setting for the GPIO pins to **no pull-up and no pull-down**.
5. Finally, set the output of the GPIO pin to have a value of 1 (corresponding to 3.3V)

Compile/build your code

- (1) Press the **build** button. Check the window at the bottom for warnings and especially for errors. The IDE will show little error icons by your code too if it detects any.
- (2) Then, plug the board into your laptop with the USB cable if you haven't already.
- (3) Now download your code to the board by pressing the **download** button.
- (4) Push the reset button (black) on the board to run your program

It is possible that your code is buggy so that Keil can't program the board anymore. In this case, you can use ST-Link utility to erase the flash memory. See this YouTube tutorial-5: <https://www.youtube.com/watch?v=OiwwB0AvlBI>



If the code doesn't work, you will have to debug your code to find out what is wrong.

Review this debug tutorial 2: <https://youtu.be/w4gPcYRk9o8>

Keil allows us to view the values of Cortex-M registers and all peripheral registers in real-time. Click the following: Peripherals, System Viewers, GPIO, and GPIOC

The screenshot displays the STM32CubeIDE interface with three main windows open:

- Registers Window:** Shows the status of various registers. The Core registers (R0-R15) are listed with their values. The PSR (Program Status Register) is set to 0x21000000. The System registers (BASEPRI, PRIMASK, FAULTMASK, CONTROL) are also shown. The Internal registers (Mode, Privilege, Stack, States, Sec) are listed. The FPU registers (S<n>, D<n>, Float, Double, FPSCR) are shown with FPSCR set to 0x03000000.
- Disassembly Window:** Shows the assembly code for the main function. The code includes comments for the LCD initialization and display functions. The main function is defined as:


```

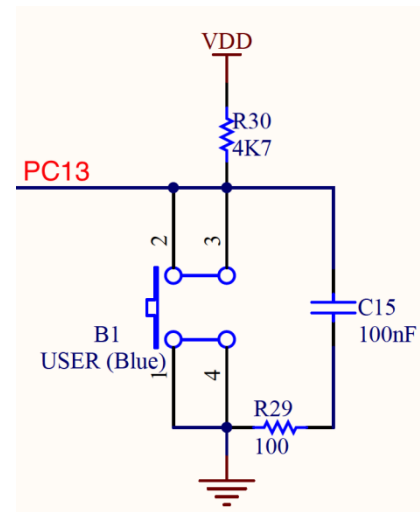
47 // SEG1
48 // SEG2
49 // SEG3
50 // SEG4
51 // SEG5
52 // USB OTG_FS
53 // OTG
54 // OTG
55 // OTG
56 // PC14 = PH0
57 // PA4 = PA3
58 // PA3 = PA3
59 // PA3 = PA3
60 // PA3 = PA3
61 // PA3 = PA3
62 // PA3 = PA3
63 // PA3 = PA3
64 // PA3 = PA3
65 void SystemInit()
66 {
67     int main()
68     {
69         LCD_Init();
70         LCD_Clear();
71         LCD_DisplayName();
72         LCD_bar();
73         while(1)
74         {
75             // Main loop
76         }
77     }
78 }
79 void SystemClock_Config()
80 {
81     // Enable HSI
82     RCC->CR |= (RCC_CR_HSION);
83     while((RCC->CR & RCC_CR_HSRDY) == 0)
84     {
85         // Adjusts the Internal High Speed oscillator
86     }
87 }

```
- GPIOA Window:** Shows the configuration for the GPIOA peripheral. The MODE register is set to 0xABEAAFF. The OTYPER register is set to 0. The OSPEEDR register is set to 0x8C2AA000. The PUPDR register is set to 0x24000000. The IDR register is set to 0x00004000. The ODR register is set to 0. The BSRR register is set to 0. The LCKR register is set to 0. The AFR registers (AFRL7, AFRL6, AFRL5, AFRL4, AFRL3, AFRL2, AFRL1, AFRL0) are set to 0x00. The AFRH register is set to 0xB0000BBB. The OSPEEDR register is set to 0x8C2AA000. The PUPDR register is set to 0x24000000. The IDR register is set to 0x00004000. The ODR register is set to 0. The BSRR register is set to 0. The LCKR register is set to 0. The AFR registers (AFRL7, AFRL6, AFRL5, AFRL4, AFRL3, AFRL2, AFRL1, AFRL0) are set to 0x00. The AFRH register is set to 0xB0000BBB.

Part B: Push Button

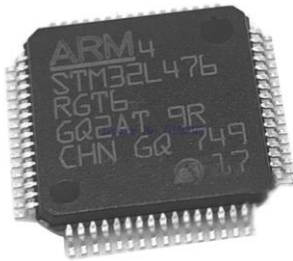
There is a blue user button the board. The button is connected to the microcontroller's **GPIO Port C Pin 13 (PC 13)**, as shown on the right.

- The pin is pulled up to VDD via a resistor (R30).
- When the button is not pressed, the voltage on PC 13 is VDD.
- When the button is pressed down, the voltage on PC 13 is 0.
- The circuit performs **hardware debouncing** using a resistor (R29) and a capacitor (C15). It forms a simple RC filter (resistive capacitive filter), which debounces the pushbutton switch.



Appendix A: Microcontroller STM32L476

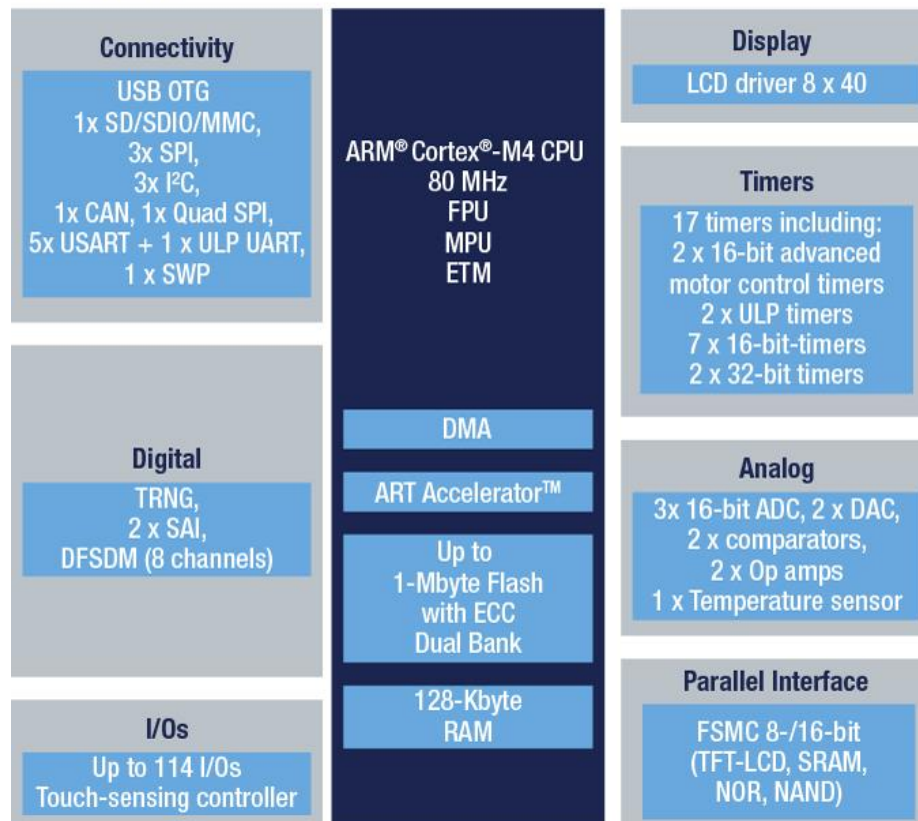
The NUCLEO-L476RG board has a 32-bit Arm Cortex-M4F microcontroller.



- Part Number: **STM32L476RGT6**
- Total number of pins: 64
- Total number of I/O pins: 51
- Maximum clock frequency: 80MHz
- Program memory size: 1 MB
- Operating supply voltage: 1.71V to 3.6V
- I/O voltage: 3.3V
- Analog supply voltage: 3.3V

The following figure summarizes the I/O connection supported and internal peripherals.

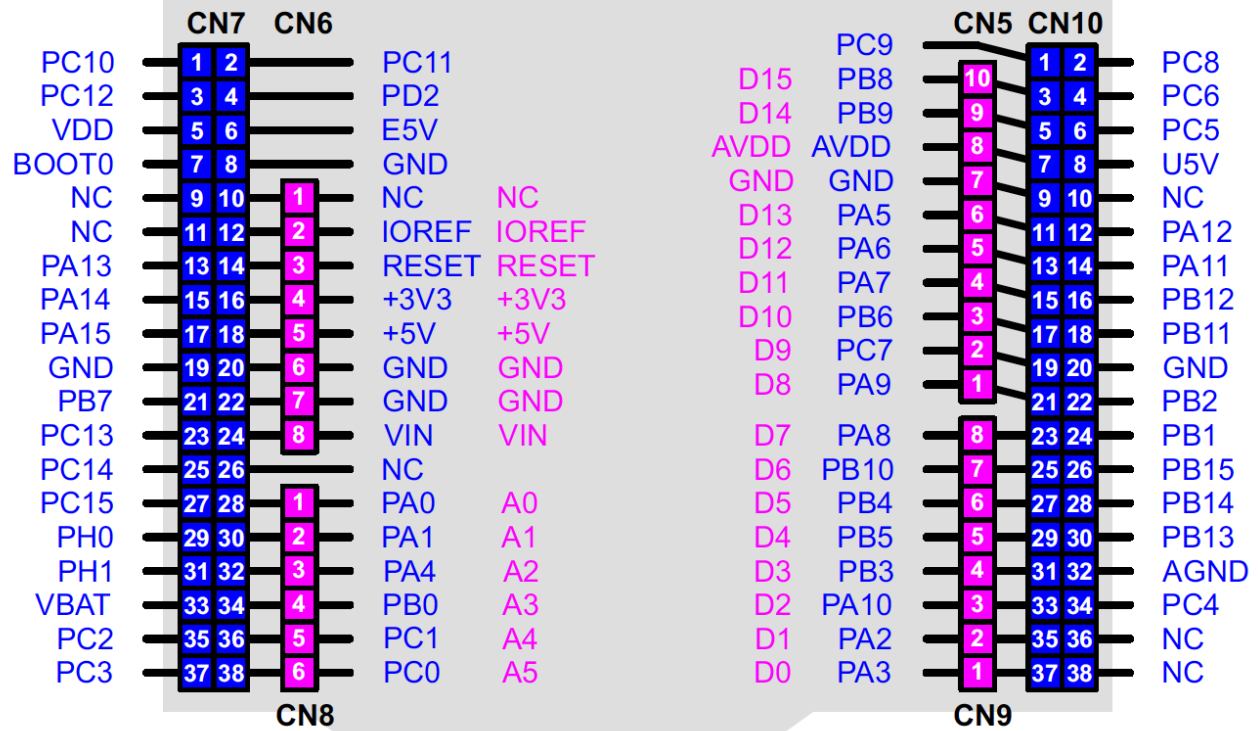
STM32L476



Appendix B: Pin Connections on Nucleo-L476RG

Board Component	Microcontroller Pin	Comment
Green LED	PA 5	SB42 closed and SB29 open by default
Blue user button	PC 13	Pulled up externally
Black reset button	NRST	Connect to ground to reset
ST-Link UART TX	PA 2	STLK_TX
ST-Link UART RX	PA 3	STLK_RX
ST-Link SWO/TDO	PB 3	Trace output pin/Test Data Out pin
ST-Link SWDIO/TMS	PA 13	Data I/O pin/Test Mode State pin
ST-Link SWDCLK/TCK	PA 14	Clock pin/Test Clock pin

NUCLEO-L476RG



Arduino

Morpho

Appendix C: Clock Configuration

There are two major types of clocks: **system clock** and **peripheral clock**. A video tutorial is given here: <https://youtu.be/o6ZWD0PAoJk>

- **System Clock:** To meet the requirement of performance and energy-efficiency for different applications, the processor core can be driven by four different clock sources, including **HSI** (high-speed internal) oscillator clock, **HSE** (high-speed external) oscillator clock, **PLL** clock, and **MSI** (multispeed internal) oscillator clock. A faster clock provides better performance but usually consumes more power, which is not appropriate for battery-powered systems.
- **Peripheral Clock:** All peripherals require to be clocked to function. However, ***clocks of all peripherals are turned off by default to reduce power consumption.***

The following figure shows the clock tree of **STM32L476**, the processor used in the STM32L4 Discovery kit. The clock sources in the domain of Advanced High-performance Bus (**AHB**), low-speed Advanced Peripheral Bus 1 (**APB1**) and high-speed Advanced Peripheral Bus 2 (**APB2**) can be switched on or off independently when it is not used. Software can select various clock sources and scaling factors to achieve desired clock speed, depending on the application's needs.

The software provided in this lab uses the 16MHz HSI as the input to the PLL clock. Appropriate scaling factors have been selected to achieve the maximum allowed clock speed (80 MHz). See the function void **System_Clock_Init()** for details.

```
void System_Clock_Init(void) {  
    ...  
  
    // Enable the Internal High Speed oscillator (HSI)  
    RCC->CR |= RCC_CR_HSION;  
    while((RCC->CR & RCC_CR_HSIRDY) == 0);  
  
    RCC->CR    &= ~RCC_CR_PLLON;  
    while((RCC->CR & RCC_CR_PLLRDY) == RCC_CR_PLLRDY);  
  
    // Select clock source to PLL  
    RCC->PLLCFGR &= ~RCC_PLLCFGR_PLLSRC;  
    RCC->PLLCFGR |= RCC_PLLCFGR_PLLSRC_HSI; // 00 = No clock, 01 = MSI, 10 = HSI, 11 = HSE
```

```

// Make PLL as 80 MHz

// f(VCO clock) = f(PLL clock input) * (PLLN / PLLM) = 16MHz * 20/2 = 160 MHz

// f(PLL_R) = f(VCO clock) / PLLR = 160MHz/2 = 80MHz

RCC->PLLCFGR = (RCC->PLLCFGR & ~RCC_PLLCFGR_PLLN) | 20U << 8;
RCC->PLLCFGR = (RCC->PLLCFGR & ~RCC_PLLCFGR_PLLM) | 1U << 4;
RCC->PLLCFGR &= ~RCC_PLLCFGR_PLLR; // 00: PLLR = 2, 01: PLLR = 4, 10: PLLR = 6, 11: PLLR = 8
RCC->PLLCFGR |= RCC_PLLCFGR_PLLREN; // Enable Main PLL PLLCLK output
RCC->CR |= RCC_CR_PLLON;
while((RCC->CR & RCC_CR_PLLRDY) == 0);

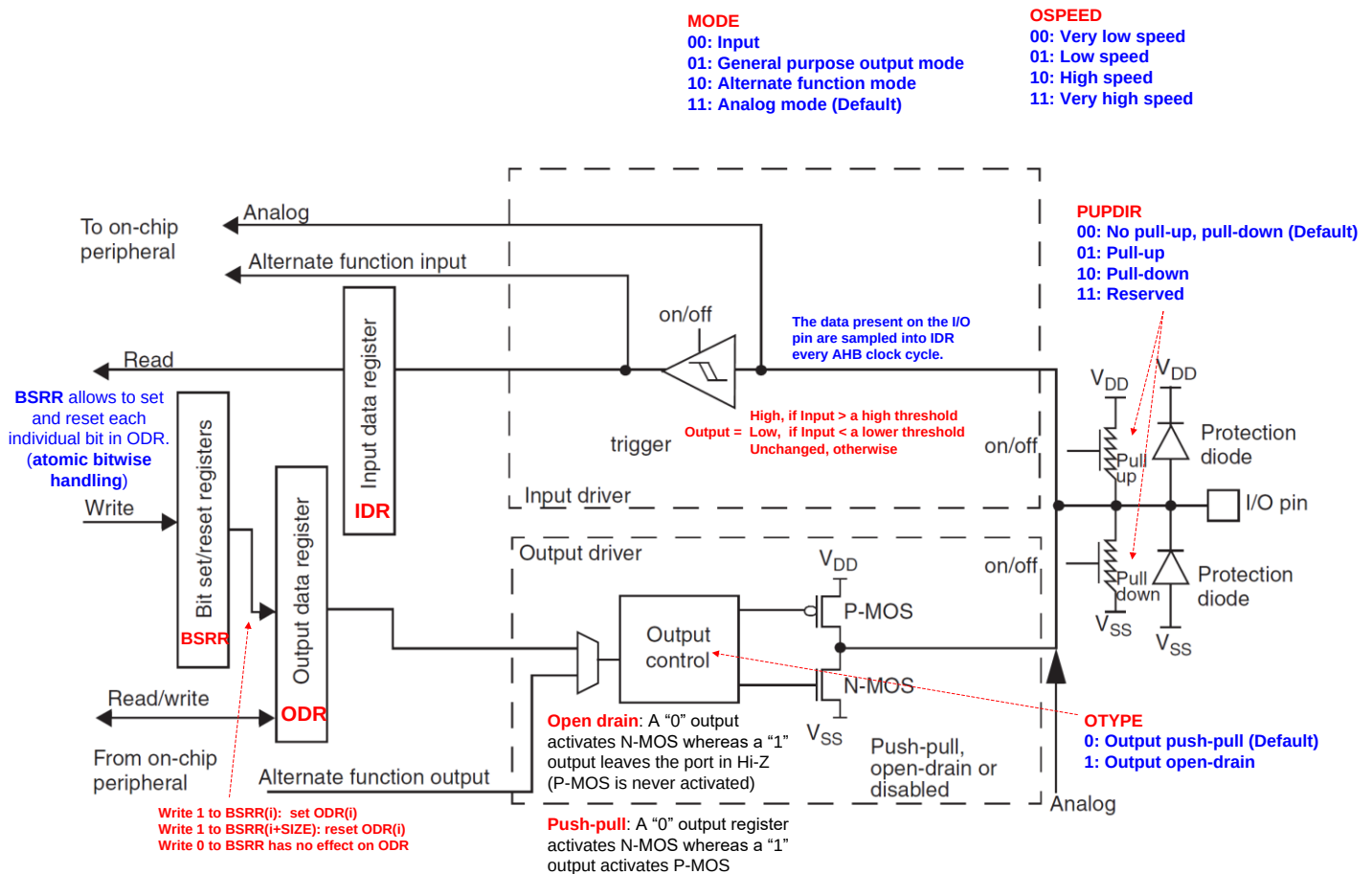
// Select PLL selected as system clock
RCC->CFGR &= ~RCC_CFGR_SW;
RCC->CFGR |= RCC_CFGR_SW_PLL; // 00: MSI, 01:HSI, 10: HSE, 11: PLL

// Wait until System Clock has been selected
while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL);
...

```


Each of the GPIO pins can be configured by software as output (push-pull or open-drain), as input (with or without pull-up or pull-down) or as peripheral alternate function. **In this lab, we will configure PB2 and PE8 as push-pull output.** Each general-purpose I/O port x (x = A, B, C, ..., H) has

- four 32-bit configuration registers
 - **MODER** (mode register)
 - **OTYPER** (output type register)
 - **OSPEEDR** (output speed register)
 - **PUPDR** (pull-up/pull-down register)
- two 32-bit data registers
 - **IDR** (input data register)
 - **ODR** (output data register)
- a 32-bit set/reset register (**BSRR**).
- a 32-bit locking register (**LCKR**)
- two 32-bit alternate function selection registers
 - **AFRH** (Alternate function high register)
 - **AFRL** (Alternate function low register)



Appendix D: Code Comments and Documentation

Program comments are used to improve code readability, and to assist in debugging and maintenance. A general principal is “***Structure and document your program the way you wish other programmers would***” (McCann, 1997).

The book titled “*The Elements of Programming Style*” by Brian Kernighan and P. J. Plauger gives good advices for beginners.

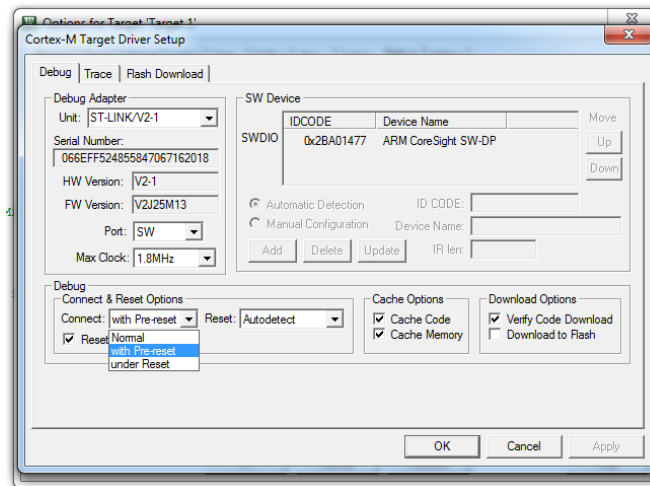
- **Format your code well.** Make sure it's easy to read and understand. Comment where needed but don't comment obvious things it makes the code harder to read. If editing someone else's code, format consistently with the original author.
- Every program you write that you intend to keep around for more than a couple of hours ought to have documentation in it. Don't talk yourself into putting off the documentation. A program that is perfectly clear today is clear only because you just wrote it. Put it away for a few months, and it will most likely take you a while to figure out what it does and how it does it. If it takes you a while to figure it out, how long would it take someone else to figure it out?
- **Write Clearly** - don't be too clever - don't sacrifice clarity for efficiency.
- **Don't over comment.** Use comments only when necessary.
- Format a program to help the reader understand it. **Always Beautify Code.**
- Say what you mean, **simply and directly.**
- **Don't patch bad code** - rewrite it.
- **Make sure comments and code agree.**
- Don't just echo code in comments - **make every comment meaningful.**
- **Don't comment bad code** - rewrite it.
- **The single most important factor in style is consistency.** The eye is drawn to something that "doesn't fit," and these should be reserved for things that are actually different.

Appendix E: "Target not found" error

When you use the STM32L4 discovery board at the very first time, you might not be able to program it in Keil and receive an error of "**Target not found**" when you download the code to the board. This is because the demo program quickly puts the STM32L4 microcontroller into a very low power mode after a reset. There are several ways to solve it. Below are two simplest ones. The error will go away permanently.

Solution 1: In Keil,

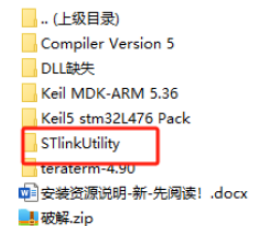
1. Click the icon "**Options for Target**"
2. Click "**Debug**" and then "**Settings**"
3. Change the connect from the default value "**normal**" to "**with pre-reset**", as shown below.



Solution 2: If the previous solution fails, you can download and install **STM32 ST-Link Utility**

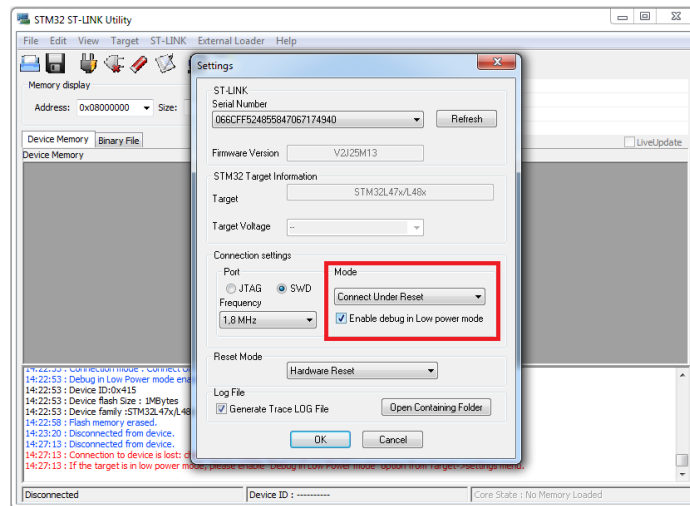
<http://www.st.com/en/development-tools/stsw-link004.html>

The STLINK Utility installation package can be found in the "Embedded" compression package, and there is no need to download



Follow the following steps:

1. Run ST-Link Utility, click menu "**Target**", click "**Settings**"
2. Select "**Connect Under Set**" as the connection mode
3. Click "**Target**" and "**Connect**", and then click "**Target**" and "**Erase Chip**"!
4. Click "**Target**" and "**Disconnect**"



Lab 2: Interfacing Push-button and LED in C

Pre-Lab Assignment

Student Name: _____

TA: _____

Time & Date: _____

Video/Textbook Reading

1. Textbook **Chapter 4.6** to review bit-wise operations and **Chapter 14 GPIO**.
2. Watch YouTube Tutorials (<http://web.eece.maine.edu/~zhu/book/tutorials.php>)
 - Lecture 5: Memory-mapped I/O (8 minutes)
 - Lecture 6: GPIO Output (11 minutes)
 - Lecture 7: GPIO Input (12 minutes)

Summary of Task: Section 1 and section 2.a show (1) how to calculate the mask and the value and (2) how to use them in C code. Follow these examples, find out the mask and value in the rest sections.

1. Enable the clock of GPIO Port A (for Green LED) and Port C (for User Button)

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
AHB2ENR														RNGEN		AESEN			ADCEN	OTGFSEN						GPIOHEN	GPIOGEN	GPIOFEN	GPIOEEN	GPIODEN	GPIOCEN	GPIOBEN	GPIOAEN
Mask																														1		1	
Value																														1		1	

In the bitmask (or called **mask**) of a variable, **1** indicates that the corresponding bit in the variable is of interest. In this example, we are interested in bit 0 and bit 2, thus we have the mask as

Mask in hex = **0x00000005**

Value in hex = **0x00000005**

Our C code is like:

```
RCC->AHB2ENR &= ~(mask); // Clear all bits of interests
```

```
RCC->AHB2ENR |= value; // Set bits of interests to target value
```

Note 1: Bit-wise Operations

Bit-wise operations update specific bits without affecting the other bits. For example, to set bit 2 of a 32-bit variable aWord, the following is incorrect because it resets all the other bits in this variable.

```
aWord = 4; // Set bit 2, but also clear all the rest 31 bits
```

The correct one is to use the bit-wise OR operation:


```
aWord |= 4; // Set bit 2 without affecting the rest
```

Note 2: Why do we need a bitmask?

A bitmask is often used for clearing the bits of interest. For example, the following code sets the least significant four bits of variable V to be 0b0101.

```
Mask = 0xF; // Mask for least significant four bits
```

```
V &= ~Mask; // Clearing bit 15 and bit 2
```

```
V |= 0x5; // Set least significant four bits to 0x0101
```

The following code, which directly uses bitwise OR without clearing the target bits first, sometimes fails. (why?)

```
V |= 0x5; // The result is not always correct.
```

2. Pin Initialization for Green LED (PA 5)

a. Set the mode of PA 5 to Output

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Mode Register (MODER)	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
Mask																					1	1										
Value																					0	1										

GPIO Mode: Input (00), Output (01), Alternate Function (10), Analog (11, default)

Mask in hex = 0x00000C00

Value in hex = 0x00000400

Example: The following C code can correctly set the mode of PA 5 to output.

```
GPIOA->MODR &= ~0x00000C00; // Clear bit 10 and bit 11
GPIOA->MODR |= 0x00000400; // Set the mode to output
```

By following the above examples, find out the mask and value in the following sections.

b. Set the output type of PA 5 to Push-Pull

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OTYPER	Reserved																OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
Mask																																
Value																																

Push-Pull (0, reset), Open-Drain (1)

Mask in hex = 0x_____

Value in hex = 0x_____

c. Set PA 5 to No pull-up No pull-down

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR	PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
Mask																																
Value																																

No Pullup, No pulldown (00, reset), Pullup (01), Pulldown (10), Reserved (11)

Mask in hex = 0x_____

Value in hex = 0x_____

3. Pin Initialization for the user pushbutton (PC 13)

a. Set the mode of pin PC 13 to Input

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Mode Register (MODER)	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
Mask																																
Value																																

GPIO Mode: Input (00), Output (01), Alternate Function (10), Analog (11, default)

Mask in hex = 0x_____

Value in hex = 0x_____

b. Set PC 13 to no pullup and no pulldown

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR	PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]		PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
Mask																																
Value																																

No Pullup, No pulldown (00, reset), Pullup (01), Pulldown (10), Reserved (11)

Mask in hex = 0x_____

Value in hex = 0x_____

Lab Demonstration Instructions

Please demonstrate that the green LED is toggled whenever the pushbutton is pressed (It can be understood that long-pressing the push button will make the LED flash).

Additionally, please demonstrate that you can toggle the LED state between on and off by directly changing the value of the ODR in the debug environment (**Peripheral → System viewer → GPIO**).

Make sure to Check “Periodic Window Update” in debug – “view”.

Lab Code Submission Instructions

Submit your code as a text file through BB, and please follow the code legibility instructions stated earlier in the lab.

Combine your solutions to the pre-lab assignment and the code as a pdf file. Submit it to Black board.